

Listas

Secuencia de elementos del mismo tipo

$[1,2,3,4] = 1:[2,3,4] = 1:2:3:[4] = 1:2:3:4:[]$

Ejemplos:

$[1, 2, 5, 7, 8] :: [\text{Int}]$

$[2, 1, 5, 7, 8] :: [\text{Int}]$

$[\text{"Hola"}, \text{"Chao"}] :: [\text{String}] \quad -- :: [[\text{Char}]]$

$[(+), (*)] :: [\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}]$

$[1, 'a', 7, 'b'] :: \text{No es una lista}$

$[(1, 'a'), (2, 'b'), (5, 'c')] :: [(\text{Int}, \text{Char})]$

$[(+), (| |), (\&\&)] :: \text{No es una lista}$

Listas

Secuencia de elementos del mismo tipo

Más Ejemplos:

[]::[a]

[[[1,2],[3,4,5],[]],[[]],[]]::

[[]] ::

[[][]] ::

[1, []] ::

[**[1, 2, 3]**, []], [[[]]]] ::

Listas

Secuencia de elementos del mismo tipo

Más Ejemplos:

$[] :: [a]$

$[[[1,2],[3,4,5],[]], [[]], []] :: [[Int]]$

$[[]] :: [[a]]$

$[[][]] :: [[[a]]]$

$[1, []] ::$ No es una lista

$[[[1, 2, 3], []], [[]]] ::$ No es una lista

porque:

$[[1, 2, 3], []] :: [Int]$

$[[]] :: [[a]]$ $[[a]]$ no puede ser $[Int]$

Funciones sobre Listas

head

head :: [a] -> a

? head [1, 2, 7, 8]

1

? head [[1, 2, 3]]

[1, 2, 3]

? head [[]]

[]

? head [],[[1, 2, 7]]

[]

Funciones sobre Listas

tail

tail :: [a] -> [a]

? tail [1, 2, 7, 8]

[2, 7, 8]

? tail [[1, 2, 3]]

[]

? tail [[]]

[]

? tail [[]]

[]

? tail []

error

? tail [1]

[]

Funciones sobre Listas

last

last :: [a] -> a

```
? last [1, 2, 3, 4]
```

```
4
```

```
? last [[1, 2, 7, 8, 9, 10]]
```

```
[1, 2, 7, 8, 9, 10]
```

```
? last [[]]
```

```
[]
```

```
? last [[][]]
```

```
[[]]
```

Funciones sobre Listas

init

init :: [a] -> [a]

? init [1, 2, 7, 8]

[1, 2, 7]

? init [[1, 2, 3]]

? init [[]]

? init [[]]

? init [[1, 2, 3], [4, 5], []]

Funciones sobre Listas

init

init :: [a] -> [a]

? init [1, 2, 7, 8]

[1, 2, 7]

? init [[1, 2, 3]]

[]

? init [[]]

[]

? init [[]]

[]

? init [[1, 2, 3], [4, 5], []]

[[1, 2, 3], [4, 5]]

Funciones sobre Listas

length

$\text{length} :: [a] \rightarrow \text{Int}$

? length [1, 2, 9, 10]

4

? length [], [2, 3], []

? length [[1, 2, 7, 9]]

Funciones sobre Listas

length

$\text{length} :: [a] \rightarrow \text{Int}$

? length [1, 2, 9, 10]

4

? length [[], [2, 3], []]

3

? length [[1, 2, 7, 9]]

1

Funciones sobre Listas

take

`take :: Int -> [a] -> [a]`

`? take 2 [7, 1, 10, 2, 4]`

`[7, 1]`

`? take 0 [1, 10, 3]`

`? take 2 [[1, 2, 7, 3, 4]]`

`? take 5 [[[True, False]]]`

Funciones sobre Listas

take

$\text{take} :: \text{Int} \rightarrow [a] \rightarrow [a]$

? take 2 [7, 1, 10, 2, 4]

[7, 1]

? take 0 [1, 10, 3]

[]

? take 2 [[1, 2, 7, 3, 4]]

[[1, 2, 7, 3, 4]]

? take 5 [[[True, False]]]

[[[True, False]]]

Funciones sobre Listas

drop

$\text{drop} :: \text{Int} \rightarrow [a] \rightarrow [a]$

? drop 2 [1, 7, 10, 2, 4]
[10, 2, 4]

? drop 0 [1, 3, 9, 8]

? drop 10 [1, 2, 3]

? drop 1 [[[1, 2, 3]]]

? drop 1 [[1, 2, 3]]

Funciones sobre Listas

drop

$\text{drop} :: \text{Int} \rightarrow [a] \rightarrow [a]$

```
? drop 2 [1, 7, 10, 2, 4]
```

```
[10, 2, 4]
```

```
? drop 0 [1, 3, 9, 8]
```

```
[1, 3, 9, 8]
```

```
? drop 10 [1, 2, 3]
```

```
[]
```

```
? drop 1 [[[1, 2, 3]]]
```

```
[]
```

```
? drop 1 [[1, 2, 3]]
```

```
[]
```

Funciones sobre Listas

zip

$\text{zip} :: [a] \rightarrow [b] \rightarrow [(a,b)]$

? zip [1, 2, 7, 9] ['a', 'b', 'c']
[(1,'a'), (2,'b'), (7,'c')]

? zip [[1, 2, 7, 8, 9]] ['a', 'b', 'c', 'd']

? zip [(>), (<)] [2, 3, 4]

Funciones sobre Listas

zip

$\text{zip} :: [a] \rightarrow [b] \rightarrow [(a,b)]$

? zip [1, 2, 7, 9] ['a', 'b', 'c']
[(1,'a'), (2,'b'), (7,'c')]

? zip [[1, 2, 7, 8, 9]] ['a', 'b', 'c', 'd']
[[([1, 2, 7, 8, 9], 'a')]

? zip [(>), (<)] [2, 3, 4]
[((>), 2), ((<), 3)]

Funciones sobre Listas

map

map ::

? map (*2) [1, 2, 7, 8]

[2, 4, 14, 16]

? map even [1, 2, 7, 8]

? map (>) [1, 2, 3, 4, 5]

? map length [[1, 2, 7], [], [3, 4]]

? map (>2) [1, 5, 7, 3, 1, 4]

? map ((>2).length) [[], [1, 2, 3], [7]]

Funciones sobre Listas

map

$\text{map} :: (a \rightarrow b) \rightarrow [a] \rightarrow [b]$

```
? map (*2) [1, 2, 7, 8]
```

```
[2, 4, 14, 16]
```

```
? map even [1, 2, 7, 8]
```

```
[False, True, False, True]
```

```
? map (>) [1, 2, 3, 4, 5]
```

```
[(>)1, (>)2, (>)3, (>)4, (>)5]
```

```
? map length [[1, 2, 7], [], [3, 4]]
```

```
[3, 0, 2]
```

```
? map (>2) [1, 5, 7, 3, 1, 4]
```

```
[False, True, True, True, False, True]
```

```
? map ((>2).length) [[], [1, 2, 3], [7]]
```

```
[False, True, False]
```

Funciones sobre Listas

filter

filter ::

? filter (>2) [1, 5, 7, 3, 1, 4]

[5, 7, 3, 4]

? filter ((>2).length) [], [1, 2, 3], [7]

? filter (>) [1, 2, 3, 4]

? filter (even.head.tail.tail) [[1, 2, 4, 5, 7], [1, 2, 3, 4], [2, 5, 6, 7]]

Funciones sobre Listas

filter

`filter :: (a -> Bool) -> [a] -> [a]`

`? filter (>2) [1, 5, 7, 3, 1, 4]`

`[5, 7, 3, 4]`

`? filter ((>2).length) [[], [1, 2, 3], [7]]`

`[[1, 2, 3]]`

`? filter (>) [1, 2, 3, 4]`

`Error (>): (a -> a -> Bool) no empareja con (Int -> Bool)`

`? filter (even.head.tail.tail) [[1, 2, 4, 5, 7], [1, 2, 3, 4], [2, 5, 6, 7]]`

`[[1, 2, 4, 5, 7], [2, 5, 6, 7]]`

Funciones sobre Listas

takeWhile

takeWhile ::

```
? takeWhile (>2) [5, 7, 1, 10, 15, 18]  
[5, 7]
```

```
? takeWhile (even.length) [[1, 2, 7, 8],[3, 9],[1],[3, 5]]
```

```
? takeWhile True [5, 10, 7, 9]
```

```
? takeWhile (>) [1, 2, 3, 4]
```

```
? takeWhile (even.length.head) [[[1, 2], [7, 9, 10]], [[3]]]
```

```
? takeWhile even [[1, 2], []]
```

Funciones sobre Listas

takeWhile

`takeWhile :: (a -> Bool) -> [a] -> [a]`

? takeWhile (>2) [5, 7, 1, 10, 15, 18]
[5, 7]

? takeWhile (even.length) [[1, 2, 7, 8],[3, 9],[1],[3, 5]]
[[1, 2, 7, 8], [3, 9]]

? takeWhile True [5, 10, 7, 9]
Error, por incompatibilidad de tipos

? takeWhile (>) [1, 2, 3, 4]
Error, porque (>)::a -> a -> Bool no empareja con : Int -> Bool

? takeWhile (even.length.head) [[[1, 2], [7, 9, 10]], [[3]]]
[[[1, 2], [7, 9, 10]]]

? takeWhile even [[1, 2], []]
Error, porque even::Int -> Bool no empareja con [Int] -> Bool

Funciones sobre Listas

dropWhile

dropWhile ::

```
? dropWhile (>2) [5, 1, 7, 10]  
  [1, 7, 10]
```

```
? dropWhile (&&True) [True, False, True]
```

```
? dropWhile ((==0).length.(takeWhile (>2))) [[1, 7, 10], [2, 8], []]
```

```
? dropWhile ((==0).length.takeWhile.>2) [[1, 7, 10], [2, 8], []]
```

Funciones sobre Listas

dropWhile

`dropWhile :: (a -> Bool) -> [a] -> [a]`

? dropWhile (>2) [5, 1, 7, 10]

[1, 7, 10]

? dropWhile (&&True) [True, False, True]

[False, True]

? dropWhile ((==0).length.(takeWhile (>2))) [[1, 7, 10], [2, 8], []]

[]

? dropWhile ((==0).length.takeWhile.>2) [[1, 7, 10], [2, 8], []]

Error, (>2) no puede aplicarse a una lista

Funciones sobre Listas

zipWith

zipWith ::

? zipWith (*) [1, 2, 3, 4] [10, 20, 30]
[10, 40, 90]

? zipWith take [1, 2, 3] [['a', 'b'], [], ['e', 'f', 'g', 'h', 'i']]

? zipWith zip [3, 4, 5] [10, 20, 30]

? zipWith zip [[3, 4, 5], []] [[True, False], [True], []]

Funciones sobre Listas

zipWith

$\text{zipWith} :: (a \rightarrow b \rightarrow c) \rightarrow [a] \rightarrow [b] \rightarrow [c]$

```
? zipWith (*) [1, 2, 3, 4] [10, 20, 30]  
[10, 40, 90]
```

```
? zipWith take [1, 2, 3] [['a', 'b'], [], ['e', 'f', 'g', 'h', 'i']]  
[['a'], [], ['e', 'f', 'g']] = ["a", [], "efg"]
```

```
? zipWith zip [3, 4, 5] [10, 20, 30]  
error (zip::[a] -> [b] -> [(a, b)]) no empareja con (Int -> Int -> [(a, b)] )
```

```
d) ? zipWith zip [[3, 4, 5], []] [[True, False], [True], []]  
[(3, True), (4, False)], []
```

Funciones sobre Listas

concat

$\text{concat} :: [[a]] \rightarrow [a]$

? concat [[1, 2], [7], [3, 9], []]
[1, 2, 7, 3, 9]

? concat [], [], [], [[]]

? concat [[1, 2, 7, 8, 9]]

? concat [[]]

? concat []

? concat [[][]]

Funciones sobre Listas

concat

$\text{concat} :: [[a]] \rightarrow [a]$

? concat [[1, 2], [7], [3, 9], []]
[1, 2, 7, 3, 9]

? concat [], [], [], []]
[]

? concat [[1, 2, 7, 8, 9]]
[1, 2, 7, 8, 9]

? concat []
[]

? concat []
[]

? concat [[]]
[]

Funciones sobre Listas

reverse

$\text{reverse} :: [a] \rightarrow [a]$

? reverse [1, 2, 3]
[3, 2, 1]

? reverse [[1, 2, 3]]

? reverse [[1, 2], []]

Funciones sobre Listas

reverse

$\text{reverse} :: [a] \rightarrow [a]$

? reverse [1, 2, 3]
[3, 2, 1]

? reverse [[1, 2, 3]]
[[1, 2, 3]]

? reverse [[1, 2], []]
[], [1, 2]

Funciones sobre Listas

resta

$(\backslash \backslash) :: [a] \rightarrow [a] \rightarrow [a]$

? [1, 2, 3, 5, 2] \ \ [2, 3, 7]
[1, 5, 2]

? [[1, 2, 3], [5, 2]] \ \ [[5, 2], [1, 3, 2], [1, 2]]
[[1, 2, 3]]

Funciones sobre Listas

unión

$(++) :: [a] \rightarrow [a] \rightarrow [a]$

? [2, 7, 8] ++ [5, 6]
[2, 7, 8, 5, 6]

? [], [2, 3], [7, 9] ++ [] ++ []
[], [2, 3], [7, 9], []

Funciones sobre Listas

Constructor de listas

$(:) :: a \rightarrow [a] \rightarrow [a]$

? 2 : [5, 7, 9]
 [2, 5, 7, 9]

? [2] : []

? [2] : [[]]

? 3 : 9 : [4]

Funciones sobre Listas

Constructor de listas

$(:) :: a \rightarrow [a] \rightarrow [a]$

? 2 : [5, 7, 9]
[2, 5, 7, 9]

? [2] : []
[[2]]

? [2] : [[]]
[[2], []]

? 3 : 9 : [4]
[3, 9, 4]

Funciones sobre Listas

Índice

$(!!) :: [a] \rightarrow \text{Int} \rightarrow a$

? [2, 7, 9, 3, 4] !! 2
9

? [[1, 2, 3, 4]] !! 2

? [[2, 7], [6], [3, 9]] !! 1

Funciones sobre Listas

Índice

$(!!) :: [a] \rightarrow \text{Int} \rightarrow a$

? [2, 7, 9, 3, 4] !! 2
9

? [[1, 2, 3, 4]] !! 2
error

? [[2, 7], [6], [3, 9]] !! 1
[6]

Funciones sobre Listas

sum, and, or

`sum :: Num a => [a] -> a`

`sum :: [Int] -> Int`

`? sum [1, 2, 3, 4]`
`10`

`and :: [Bool] -> Bool`

`? and [True, False, False]`
`False`

`? and []`
`True`

`or :: [Bool] -> Bool`

`? or [False, True, False]`
`True`

`? or []`
`False`

Funciones estándar sobre listas

Ejercicios

Obtener la fila n de una matriz

```
fila :: [[a]] -> Int -> [a]
```

```
fila xss n = head (drop (n-1) xss)
```

```
fila2 xss n = xss!!(n-1)
```

```
? fila [[1,2,3],[11,22,33],[10,20,30]] 2 => [11,22,33]
```

Obtener la columna n de una matriz

```
col :: [[a]] -> Int -> [a]
```

```
col xss n = map (!! (n-1)) xss
```

```
? col [[1,2,3],[11,22,33],[10,20,30]] 2 => [2,22,20]
```

Verificar si una matriz es un cuadrado perfecto